DICOM Duplicate-Upload Detection

Review of unstaged working-tree changes • care_radiology • branch `main` • 18 September 2026

Files changed

File	Change	Lines
`src/care_radiology/utils/dicom.py`	New `read_sop_instance_uid()` DICOM meta-group parser	+56
`src/care_radiology/services/dicom_service.py`	Pre-upload duplicate guard in `upload_dicom_file()`	+13

The problem being fixed

A DICOM file's identity is its SOP Instance UID, not its filename. PACS stores instances keyed on that UID, and `DicomStudy.objects.update_or_create()` keys on the study UID. Re-uploading a file that was already stored therefore overwrote it in place and still returned **201 Created** — the caller was told the upload succeeded while nothing had actually changed. The new code rejects the re-upload up front so the caller is told it is a duplicate.

1. New parser: `read_sop_instance_uid(file_obj)`

src/care_radiology/utils/dicom.py

Extracts `MediaStorageSOPInstanceUID (0002,0003)` from a DICOM Part 10 file's meta group. That value is identical to the dataset's SOPInstanceUID and uniquely identifies the single file.

- Validates the 128-byte preamble followed by the `DICM` magic; returns `None` if the file is not a parseable Part 10 file.
- Walks the meta group tag by tag reading 8-byte headers, and handles the 12-byte long-form VRs (`OB, OW, OF, SQ, UT, UN`) by reading the extra 4-byte length field.
- Stops as soon as `group != 0x0002` — past the meta group the transfer syntax may change, and the wanted UID is always inside it.
- **No DICOM library needed.** The meta group is always explicit-VR little-endian regardless of the dataset's transfer syntax, so it can be parsed directly.
- Strips the trailing null byte UIDs are padded with, and returns `None` on `OSError`, `UnicodeDecodeError` or `ValueError`.
- A `finally` block calls `file_obj.seek(0)` so the subsequent `encode_file_multipart_related()` still reads the file from the start.

```
_FILE_META_LONG_FORM_VRS = (b"OB", b"OW", b"OF", b"SQ", b"UT", b"UN")
FILE_META_GROUP = 0x0002
MEDIA_STORAGE_SOP_INSTANCE_UID = (0x0002, 0x0003)
```

2. Pre-upload duplicate guard

src/care_radiology/services/dicom_service.py

Added at the very top of `upload_dicom_file()`, before anything is sent to PACS:

```
sop_instance_uid = read_sop_instance_uid(dcm_file)
if sop_instance_uid and d_query_instance(sop_instance_uid) is not None:
    raise DicomUploadError(
        "Duplicate : File already uploaded.",
        status_code=409,
        extra={"duplicate": True, "sop_instance_uid": sop_instance_uid},
    )
```

- The caller now receives **409 Conflict** instead of a misleading 201, with `extra={"duplicate": True, "sop_instance_uid": ...}` for the client to act on.
- The check happens before `encode_file_multipart_related()`, so a duplicate costs one lightweight PACS query rather than a full multipart upload.
- The existing `except DicomUploadError: raise` at the end of the function passes the error through untouched, so the broad `except Exception` handler does not swallow it into a generic 500.

Two behavioural notes worth confirming

Fails open on PACS trouble

`d_query_instance()` returns `None` when the PACS response is not OK or the JSON is invalid. If PACS is down or degraded, the duplicate check silently passes and the upload proceeds. That is the safer direction — an outage does not block legitimate uploads — but it means the guard is best-effort, not a hard constraint. Genuine duplicates can still get through during an incident.

The check is global, not per-patient

The query is `SOPInstanceUID=...` across the whole PACS with no patient filter, so uploading the same file under a *different* patient is also rejected with 409. This is defensible — SOP Instance UIDs are globally unique, so one instance genuinely should not belong to two patient records — but if any workflow re-uses instances across patients it would be blocked. Worth confirming that is intended.

Both changes are unstaged in the working tree at the time of review. No tests accompany them; `read_sop_instance_uid()` is a byte-level parser and is a good candidate for unit tests covering a valid Part 10 file, a long-form VR, a truncated file and a non-DICOM file.